

04 · Bag of Words (BoW)

Bag of Words representa cada documento como un vector con la **frecuencia** de cada palabra del vocabulario, **ignorando el orden**.

Como "documentos" usamos las **descripciones de producto** del catálogo (`datos/productos.csv`): 90 descripciones repartidas en 9 categorías, un corpus con vocabulario técnico variado.

```
from pathlib import Path

import pandas as pd

# El texto ya minado de la tienda-virtual está copiado en la carpeta datos/ de
# este mismo proyecto:
# datos/resenas_entrega.csv   reseñas de entrega (post_compra)
# datos/comentarios.csv      testimonios / comentarios de clientes
# datos/productos.csv        catálogo con la descripción de cada producto
DATOS = Path("../datos")
```

```
def cargar(nombre, **kwargs):
    """Lee un CSV de la carpeta datos/ y lo devuelve como DataFrame."""
    ruta = DATOS / nombre
    if not ruta.exists():
        raise FileNotFoundError(f"No se encontró {ruta.resolve()}")
    print(f"Leyendo {ruta} ({ruta.stat().st_size / 1024:.1f} KB)")
    return pd.read_csv(ruta, **kwargs)
```

```
productos = cargar("productos.csv")
print(productos.shape)
productos[["nombre", "categoria", "descripcion"]].head(3)
```

```
Leyendo ../datos/productos.csv (27.5 KB)
(90, 11)
```

	nombre	categoria	descripcion
0	Apple iPhone 15 Pro Max 256GB	Smartphones	Smartphone premium con chip A17 Pro fabricado ...
1	Apple iPhone 15 128GB	Smartphones	Smartphone con chip A16 Bionic, camara dual de...
2	Samsung Galaxy S24 Ultra 512GB	Smartphones	Smartphone con S Pen integrado, camara cuadруп...

Normalización (mismo pipeline de 02_normalizacion.ipynb)

```
import re
import unicodedata
```

```

import nltk
import spacy

nlp = spacy.load("es_core_news_sm")
STOPWORDS = set(nltk.corpus.stopwords.words("spanish"))

def _limpiar(texto):
    # NFKC junta tildes combinantes sueltas; luego minúsculas y solo letras/espacios
    texto = unicodedata.normalize("NFKC", str(texto)).lower()
    return re.sub(r"^\w\s", " ", texto)

def _lemas(doc):
    return [
        t.lemma_.lower()
        for t in doc
        if t.is_alpha and not t.is_stop and t.lemma_.lower() not in STOPWORDS and
        len(t.lemma_) > 2
    ]

def normalizar(texto):
    """minúsculas -> sin signos -> sin stopwords -> lematizado. Devuelve un str."""
    return " ".join(_lemas(nlp(_limpiar(texto))))

def normalizar_muchos(textos):
    """Igual que normalizar() pero en lote con nlp.pipe (más rápido para un corpus)."""
    return [" ".join(_lemas(doc)) for doc in nlp.pipe(_limpiar(t) for t in textos),
    batch_size=64)]

```

```

docs = productos["descripcion"].fillna("").tolist()
docs_norm = normalizar_muchos(docs)
print("original  :", docs[0][:110], "...")
print("normalizado:", docs_norm[0][:110], "...")

```

original : Smartphone premium con chip A17 Pro fabricado en 3nm, camara triple de 48MP con teleobjetivo periscopico de 5x ...
 normalizado: smartphonir premium chip pro fabricado camamar triple teleobjetivo periscopico pantalla super retín xdr pulgad ...

Nota sobre el ruido del preprocesamiento. El modelo `es_core_news_sm` a veces produce lemas extraños en términos de dominio o en texto sin tildes (`smartphone` → `smartphonir`, `retina` → `retín`). Es un recordatorio de que ningún pipeline es perfecto: para producción se usan modelos más grandes (`es_core_news_md/lg`) o listas de excepciones.

Vocabulario y matriz de conteos

`CountVectorizer` construye el vocabulario y cuenta. Con `min_df=2` descartamos palabras que aparecen en un solo documento (ruido).

```

from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer(min_df=2)
X = vectorizer.fit_transform(docs_norm)
vocab = vectorizer.get_feature_names_out()

print(f"Matriz BoW: {X.shape[0]} documentos x {X.shape[1]} términos del vocabulario")

df_bow = pd.DataFrame(X.toarray(), columns=vocab, index=productos["codigo"])
df_bow.iloc[:5, :14]

```

Matriz BoW: 90 documentos x 253 términos del vocabulario

	abatible	acceso	accion	actividad	adaptativo	adicional	agua	ajustar	alexa	almacena
codigo										
PROD-001	0	0	0	0	0	0	1	0	0	
PROD-002	0	0	0	0	0	0	0	0	0	
PROD-003	0	0	0	0	0	0	0	0	0	
PROD-004	0	0	0	0	0	0	0	0	0	
PROD-005	0	0	0	0	0	0	0	0	0	

Dimensionalidad y dispersión (*sparsity*)

```

total = X.shape[0] * X.shape[1]
no_cero = X.nnz
print(f"Celdas totales      : {total}")
print(f"Celdas distintas de 0: {no_cero}  ({100 * no_cero / total:.1f} %)")
print(f"Dispersión (sparsity): {100 * (1 - no_cero / total):.1f} % de la matriz son ceros")
print(f"Términos por documento: {no_cero / X.shape[0]:.1f} de {X.shape[1]} en promedio")

```

Celdas totales : 22770
 Celdas distintas de 0: 1057 (4.6 %)
 Dispersión (sparsity): 95.4 % de la matriz son ceros
 Términos por documento: 11.7 de 253 en promedio

Palabras más frecuentes por categoría

Sumando las filas BoW de cada categoría vemos que el vocabulario dominante es **temático**: BoW captura de qué trata un texto aunque pierda el orden.

```

df_cat = df_bow.copy()
df_cat["categoria"] = productos["categoria"].values
sumas = df_cat.groupby("categoria").sum()

for cat in ["Smartphones", "Audio", "Fotografia y Video"]:

```

```
top = sumas.loc[cat].sort_values(ascending=False).head(8)
print(f"\n{cat}:")
print("  " + ", ".join(f"{w}({int(n)})" for w, n in top.items()))
```

Smartphones:

pantalla(11), smartphonir(10), pulgada(9), carga(6), amoled(6), camara(5), super(4), rapido(4)

Audio:

hora(10), ear(6), bateria(5), cancelacion(5), ruido(5), portatil(4), reproduccion(4), parlante(4)

Fotografia y Video:

camarar(5), pantalla(4), aps(4), camara(4), autoenfoco(3), ideal(3), incluir(3), video(3)

Limitación: BoW ignora el orden

Dos frases con las mismas palabras en distinto orden producen **el mismo vector**.

```
par = ["el cargador daña la batería", "la batería daña el cargador"]
vec_par = CountVectorizer().fit(par)
Xpar = vec_par.transform(par).toarray()
demo = pd.DataFrame(Xpar, columns=vec_par.get_feature_names_out(), index=["frase A",
"frase B"])
print(demo)
print("\n¿Vectores idénticos?", bool((Xpar[0] == Xpar[1]).all()))
```

	batería	cargador	daña	el	la
frase A	1	1	1	1	1
frase B	1	1	1	1	1

¿Vectores idénticos? True

Siguiente: `05_tf-idf.ipynb` — pondera cada término por lo distintivo que es, no solo por cuántas veces aparece.